



COMP4141 Slides Week 9

Brendan Mabbutt

UNSW

April 13, 2026



UNSW
SYDNEY

Admin

My Experience

MyExperience is open for you to leave your feedback so that this course can improve in future iterations. Please fill this out!

Due Dates

Due date for the week 8 homework has been moved from Monday, April 13 back to Friday, April 17. The final homework for week 9 will similarly be due on the Friday of week 10.

Exam Dates

The exam is 9th of may.

In class assessment this week

Only 15 minutes this week.

coNP



coNP

We don't know if $NP = coNP$

coNP

coNP is the set of all languages $\Sigma^* \setminus A$, where A is a language in NP.

$$P \subseteq (coNP \cap NP)$$

$$P \subseteq coNP$$

NP $\rightarrow$ can check right answers in PTIME

coNP $\rightarrow$ can check incorrect answers in PTIME

coNP-completeness

coNP-completeness

A language L is coNP-complete if

- $L \in \text{coNP}$, and
- every problem in coNP is polynomial-time reducible to L .

It is unknown whether $\text{NP} = \text{coNP}$.

PSPACE

PSPACE and NPSPACE

Non-deterministic

PSPACE and NPSPACE

Let $f : \mathbb{N} \rightarrow \mathbb{N}$. Define the space complexity classes $\text{SPACE}(f(n))$ and $\text{NPSPACE}(f(n))$ as the collections of all languages decidable by a TM using $O(f(n))$ space and an NTM using $O(f(n))$ space, respectively.

Example: SAT

SAT Space Complexity

$\text{SAT} \in \text{SPACE}(n)$

PSPACE and NPSPACE

$\text{PSPACE} = \bigcup_{k \in \mathbb{N}} \text{SPACE}(n^k)$ and $\text{NPSPACE} = \bigcup_{k \in \mathbb{N}} \text{NSPACE}(n^k)$.

SPACE vs. NSPACE

Savitch's Theorem

For any function $f : \mathbb{N} \rightarrow \mathbb{N}$ with $f(n) \geq n$,

$$\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2).$$

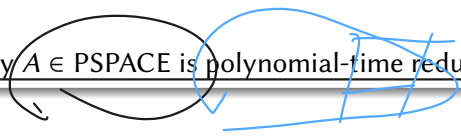
Corollary

$\text{PSPACE} = \text{NPSPACE}.$

PSPACE-Completeness

PSPACE-Completeness

A language B is PSPACE-complete if

1. $B \in \text{PSPACE}$, and
 2. B is PSPACE-hard, i.e., every $A \in \text{PSPACE}$ is polynomial-time reducible to B .
- 

Quantified Boolean Formulas

Quantified Boolean Formulas

$$\varphi \rightarrow p \mid \varphi \wedge \varphi \mid \neg \varphi \mid (\varphi) \mid \forall p(\varphi) \mid \exists p(\varphi)$$

$$\forall x(\varphi) = \varphi[\text{true}/x] \wedge \varphi[\text{false}/x], \quad \exists x(\varphi) = \varphi[\text{true}/x] \vee \varphi[\text{false}/x]$$

$$\text{QBF} = \{\langle \varphi \rangle \mid \varphi \text{ is true}\}$$

QBF Space Complexity

QBF is PSPACE-complete.

Two-Player Games and PSPACE-Completeness

Complexity of Two-Player Games

- Let ψ be a Boolean formula over propositions $x_1, \dots, x_k$. In move i , a player assigns a truth value to x_i . The problem

$$\{\langle \psi \rangle \mid E \text{ has a winning strategy}\}$$

is **PSPACE**-complete.

- Geography Game is **PSPACE**-complete.
- Deciding whether a given $n \times n$ Go position is winning is **PSPACE**-hard.

The question: “Does player E have a winning strategy from position p in game G ?” can often be expressed as a quantified Boolean formula:

$$\exists m_1 \forall r_1 \exists m_2 \cdots \varphi.$$

L and NL

L and NL

Definition

$L = \text{SPACE}(\log n)$, $NL = \text{NSPACE}(\log n)$.

Example of NL

S-T REACHABILITY

Input: A directed graph G and vertices s, t

Question: Is t reachable from s in G ?

Log-Space Reducibility

Definition

A log-space transducer is a 3-tape TM with:

- a read-only input tape,
- a read-write work tape, and
- a write-only output tape,

that uses only $O(\log n)$ space on the work tape.

We use log-space reducibility ($\leq_L$) based on such transducers.

NL-completeness

NL-completeness

A language B is NL-complete if $B \in \text{NL}$ and for every $A \in \text{NL}$ we have $A \leq_L B$.

Theorem

S-T REACHABILITY is NL-complete.

coNL and NL

Theorem (Immerman–Szelepcsényi)

$NL = coNL$.

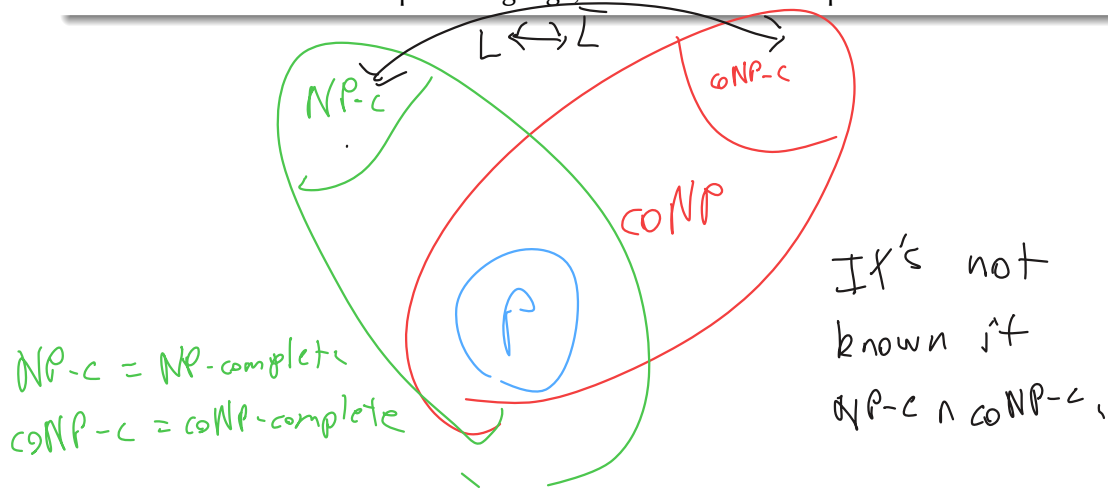
Proof idea: show that the complement of s-t REACHABILITY is in NL.

$A \in coNL$ if $\bar{A} \in NL$

W9 P1

Problem 1

Show that if L is an **NP**-complete language, then $\bar{L}$ is **coNP**-complete.



Problem 1

Show that if L is an **NP**-complete language, then $\bar{L}$ is **coNP**-complete.

Want to show

1) $\bar{L} \in \text{coNP}$

2) $\bar{L}$ is coNP-complete

For ① since $L \in \text{NP-c}$ so that $L \in \text{NP}$
 $\Rightarrow \bar{L} \in \text{coNP}$

$\downarrow$
 L is also
NP-hard

For (2) we want to show that $\forall K \in \text{coNP}$
we have that $K \leq_p \bar{L}$

(to show $\bar{L}$ coNP-hard)

Since L is NP-hard $\forall J \in \text{NP}$

$$J \leq_p L$$

$\Rightarrow \forall J \in \text{NP}, \exists f$ computable

s.t. $w \in J$ iff $f(w) \in L$

then $w \notin J$ iff $f(w) \notin L$
 $w \in \overline{J}$ iff $f(w) \in \overline{L}$
 $(\overline{J} \leq_p \overline{L})$

(In general) $A \leq_p B \iff \overline{A} \leq_p \overline{B}$

Since $\overline{K} \in \text{coNP}$, $\overline{K} \in \text{NP}$ so

$\overline{K} \leq_p L$ so $K \leq_p \overline{L}$

So that any problem in coNP

So $\bar{L}$ is coNP hard so since

it is also in coNP , it is

coNP -complete.

W9 P2

Problem 2

Show that the following problem is **coNP**-complete:

Validity

Input: A Boolean formula φ

Question: Is φ true for all truth assignments?

SAT asked for φ to true
for at least one.

1) Want to show Validity is in
coNP. We can show Validity $\in$ NP,

Validity asks if ϕ is not
true for at least one assignment

→ Verifier will take a
certificate as an assignment
and checks that the assignment
is satisfying in $O(|\phi|)$ time

Assumes
we can
verify if
w is a boolean
formula in PTIME
if true; else (Y/N)

② To prove Validity in coNP ,

We want show $\overline{\text{Validity}}$ is NP -c
by Problem 1

We will show $\text{SAT} \leq_p \overline{\text{Validity}}$

We will use $f(x) = \neg x$

Then $\varphi \in \text{SAT} \iff \exists \underline{x} \text{ s.t. } \varphi(\underline{x}) = \text{true}$
 $\iff \exists \underline{x} \text{ s.t. } \neg \varphi(\underline{x}) = \text{false}$
 $\iff \neg \varphi = f(\varphi) \in \text{Validity}$

• f can be computed in PTIME

so by problem, since SAT is NP-C,

Validity is coNP-C

Since we know Validity
is also in coNP.

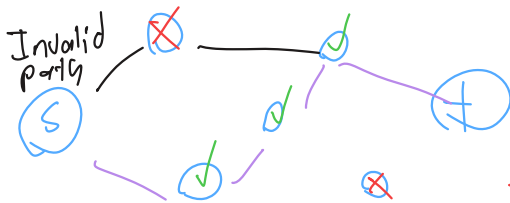
W9 P3

Problem 3

A ladder in a language L is a sequence of strings in L , all of the same length, such that each differs from the preceding one by at most one character. For example, the following is a ladder in English, starting with “head” and ending in “free”.

head, hear, near, fear, bear, beer, deer, deed, feed, feet, fret, free.

Let $LADDER_{DFA} = \{\langle M, s, t \rangle \mid M \text{ is a DFA and } L(M) \text{ contains a ladder from } s \text{ to } t\}$. Show that $LADDER_{DFA} \in \mathbf{PSPACE}$.



Valid if
all words
only differ
at one position

x not in L
✓ is in L

Consider language
assume $|S| = |T|$

Σ

(since we
would reject)
otherwise

Our sequence

$S, w_1, w_2, \dots, t$

We can assume
(cannot itself does not
otherwise remove
them)

then the number of words
in the sequence is

$$|\Sigma|^{|\mathcal{S}|}$$

(not that
the input
size $\Omega(|\mathcal{S}| + \log(|\Sigma|))$)

↑
Express
s and t

↑
comes
from
the DFA

• The idea of an algorithm would be guess/try all possible sequence but that would have a stack of size $O(|S| |\epsilon|^{|S|})$ which is not polynomial

n could be $\approx \log |\epsilon|$
then $e^{\alpha(n)}$ could be (ϵ)

But since $PSPACE = NPSPACE$
we can show $LADDER \in NPSPACE$

Our WTM works as follows:

"On input $\langle M, s, t \rangle$:"

if $|s| \neq |t|$ - reject

let $w := s$

and Σ be the alphabet of M

For $| \Sigma |^{|S|}$ iterations

From w guess a
valid next word in
the sequence (call w')

(Need to check $w' \in L(M)$
by running w' on M
which is $O(|S|)$)

If $w' = \perp$ then we accept

Guess a
position
and a different
character

(if the for loop ends)
Reject

• Want to show this runs
in polynomial space

• Each run on the DFA does not require
more space

• we need $\log |\Sigma|^{|s|} = |s| \log |\Sigma|$ space to store
which iteration we are on $\uparrow$ polynomial in the input size

" to store w would be $O(|\Sigma| \lg |\Sigma|)$

so this a polynomial space
non deterministic algorithm that
decides $L(A) \cap D(E)$ DFA.

$\Downarrow$ $\in PSPACE$

$\subseteq NPSPACE(E)$

W9 P4

Problem 4

Show that the following problem is **PSPACE**-complete:

The input size is $\Omega(k)$.

Linear-bounded DTM acceptance (LBDA)

Input: A Turing Machine M and a word w and a natural number k (in unary)

Question: Does M accept w using at most $k|w|$ tape cells?

2) To show LBDA is in PSPACE:

• On input w , run M on w and reject if more than $k|w|$ cells are used so this would require linear space

2) To show LBDA is PSPACE-hard,
take any $L \in \text{PSPACE}$ and show
 $L \leq_p \text{LBDA}$

Since L is in PSPACE there is
a t.m. M s.t. $L(M) = L$
and M runs in space $C \cdot n^2$

We want roughly $k |f(n)| \approx c \cdot n^2$

one way is with $k=1$
and $|w'| \approx c \cdot n^2$

Given w
we compute

(want $w \in L$ (w is accepted by M)
iff $f(w) \in LBPA$)

$$f(w) = \langle M', w', 1 \rangle$$

$w' = w \# 0^{(|w|^2 + c)}$ ← Extra space to simulate any Turing machine
 (pad the string)

$M' = 1$ on input x

if x not on the form
 $w \# 0^m$ reject

otherwise M on w and
 accept if M accepts

$w \in L = L(M)$ iff w is accepted
in $C \cdot |w|$ space

iff w is accepted by
 M' in space $C \cdot |w|^p + C$

which is at most

$$k |w'|$$

$(k \geq 1, w' = w \# 0^{C \cdot |w|^p + C})$

W9 P5

Problem 5

Consider the following decision problem

3-SAT is NP-c

2-SAT

Input: A boolean formula φ in 2-CNF (at most 2 literals per clause)

Question: Is φ satisfiable?

(a) Show that 2-SAT $\in$ P

$$\varphi = (x_a \vee x_b) \wedge (\neg x_a \vee x_c) \wedge \dots$$

Logically $\alpha_a \vee \alpha_b$

is equivalent to

$$\neg \alpha_a \rightarrow \alpha_b$$

and

$$\neg \alpha_b \rightarrow \alpha_a$$

G

$x_a \vee \neg x_b$
equivalent to

$\neg x_a \rightarrow \neg x_b$
 $x_b \rightarrow x_a$

x_a

x_b

x_c

2 * |clauses|

number
of edges
(m)

$\neg x_a$

$\neg x_b$

$\neg x_c$

2 * |variables| = # nodes
(n)

So we can construct this in PTIME

If you consider a path

$$y_u \Rightarrow \neg y_v \Rightarrow \dots y_w$$

then $y_u = \neg y_v = y_w$
(either all true)
or all false)

So if we have a path

$$y_u \rightarrow \dots \rightarrow y_v$$

then this is a contradiction

you can check by a BFS
if such a loop exists from
each node in time $O(n^2)$

If no such cycle exist
then φ is satisfiable

Problem 5

Consider the following decision problem

2-SAT

Input: A boolean formula φ in 2-CNF (at most 2 literals per clause)

Question: Is φ satisfiable?

(b) Show that $2\text{-SAT} \in \text{NL}$

'By $\exists$ thm we can show
 $2\text{-SAT} \in \text{NL}$

'Can come up with an algorithm
guesses a path from y_u to $\neg y_u$

W8 P5

Problem 5

Consider the following problem:

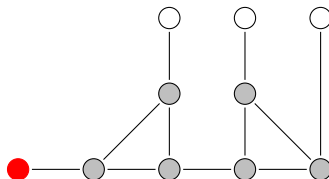
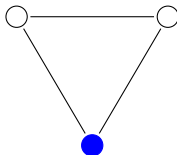
3-CoL

Input: An undirected graph G

Question: Is there a colouring of the vertices of G using at most 3 colours such that no edge has both endpoints with the same colour?

Problem 5

- (a) What can be said about the colouring of the white vertices in the following (partially coloured) graphs?



Problem 5

(b) Show that 3-CoL is **NP**-complete, assuming that SAT is **NP**-complete.

W8 P6

Problem 6

Show that if $\mathbf{P} = \mathbf{NP}$, then every language $A \in \mathbf{P}$ except $A = \emptyset$ and $A = \Sigma^*$, is **NP**-complete.